Belief: What the Robot Knows

Contents

1. Overview . . . 2
2. What the robot knows: its history . . . 3
   - 2.1 Why the true cell is out of reach . . . 3
   - 2.2 The information state: everything the robot knows . . . 4
3. Summary 1: the set of cells the robot could be in . . . 4
   - 3.1 The set, step by step . . . 4
   - 3.2 Why a set is not enough . . . 6
4. Summary 2: a probability for every cell . . . 6
   - 4.1 The belief, as a picture . . . 6
   - 4.2 The definition . . . 8
   - 4.3 A belief is not a position . . . 8
   - 4.4 Why the whole history appears in the definition . . . 8
   - 4.5 The predicted belief: after the move, before the reading . . . 9
5. The two tables that drive the belief . . . 10
   - 5.1 How a command moves the robot: the state transition probability . . . 10
   - 5.2 What the sensor reads in each place: the measurement probability . . . 12
6. The whole picture: the hidden Markov model . . . 13
   - 6.1 A small piece of maths: Bayesian networks . . . 13
   - 6.2 The robot as a dynamic Bayes network . . . 14
   - 6.3 Hidden Markov model . . . 14
7. Summary . . . 16
8. Sources . . . 16
9. Key terms . . . 17

Where this fits:

- **Builds on:** Conditional probability; Probability distributions; State, controls and measurements; Probabilistic motion models; Beam model.
- **Leads to:** Bayes filter; Smoothing and the Viterbi algorithm.

1. Overview

Key point: A robot never sees its own position. What it can keep is a **belief** (G-2433): a probability for every place it might be, computed from the commands it gave and the readings it got.

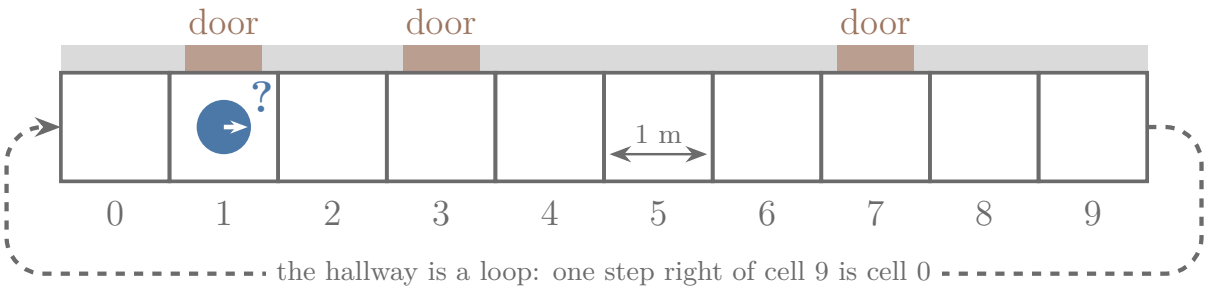


Figure 1: A hallway of 10 cells, numbered 0 to 9 and joined into a loop, with doors in front of cells 1, 3 and 7; a robot stands in cell 1 and does not know it

With the landmark model, the robot has a **motion model** (G-2319) and sensor models, so it can score any guessed pose for one command or one reading. What is still missing: each score covers one moment, and nothing says what the robot should carry from one step to the next. This Note adds that memory, the belief.

Figure 1 shows the world of this Note: the same hallway of 10 cells as before, with the same layout and numbers, because a belief over 10 cells is 10 numbers we can draw and check by hand. A robot stands in a hallway of 10 cells, each 1 m long. The hallway is a loop: one step to the right of cell 9 is cell 0. There are doors in front of cells 1, 3 and 7, and all three look the same.

The robot has two sources of information:

- a **door sensor** that reads "door" or "wall", and is sometimes wrong;
- its **commands**, such as "move 2 cells to the right", which its wheels carry out only roughly.

The robot stands in cell 1, but nothing tells it so. A reading of "door" fits three cells. A move of "2 cells" may really be 1 or 3. So the question of this Note is: **what should the robot keep in memory about where it is, so that it can act sensibly?**

We answer it in steps:

- what the robot really knows: the list of its commands and readings (Section 2);
- a first summary of that list: the set of cells it could be in, and why a set is not enough (Section 3);
- a better summary: a probability for every cell, the belief (Section 4);
- the two probability tables that tell the robot how commands and readings change the belief (Section 5);
- one picture of how states, commands and readings depend on each other, the **hidden Markov model** (G-2439) (Section 6).

The **state** (G-2314) is what the robot keeps track of, and the commands and readings are the two streams of data it receives. In symbols:

- t is the time step, counted 1, 2, 3;
- x_t is the state, the robot's cell at time t, a cell number from 0 to 9 (cell 1 at $t = 1$);
- u_t is the command at time t, such as "move 2";
- z_t is the reading at time t, "door" or "wall".

2. What the robot knows: its history

> **Key point:** The robot cannot know its true cell. It can only know the commands it gave and the readings it got; that list is its **information state** (G-2431).

2.1 Why the true cell is out of reach

The robot stands in cell 1, but its sensor reads "door" in front of three different cells, and it is sometimes wrong. The wheels slip, so the command "move 2" does not say exactly how far the robot went. Nothing the robot has ever received names its cell. Figure 2 sorts the pieces into what the robot cannot see (the top row) and what it can (the framed rows).

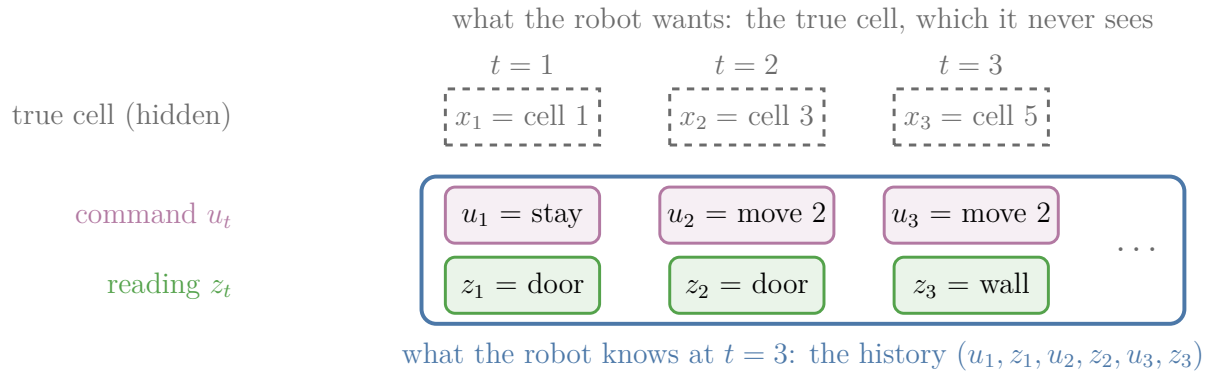


Figure 2: Top row, hidden: the true cells x1 = 1, x2 = 3, x3 = 5. Framed rows, known: the commands stay, move 2, move 2 and the readings door, door, wall. The frame is the history the robot knows at t = 3

The robot's run in Figure 2 is the running example of this Note:

Time t	Command u_t	Reading z_t	True cell x_t (hidden)
1	stay	door	1
2	move 2	door	3
3	move 2	wall	5

2.2 The information state: everything the robot knows

The framed part of Figure 2 is everything the robot knows at time t: what it knew at the start, followed by every command and every reading so far. This list is the **information state** (G-2431), also called the history information state (LaValle §11.1.2). At time 2 it is the list (stay, door, move 2, door). Each new step adds one command and one reading.

Why give the list a name? Because it is the only thing a robot's decisions can be based on. A rule such as "if you think you are at the second door, open it" has to be a rule on what the robot knows, not on its true cell, which it never sees (LaValle §11.1.3).

The drawback of the list is its length. The list grows by two entries every step and never shrinks. A robot that reads its sensor 10 times a second has, after one hour:

$$10 \times 3600 = 36{,}000 \text{ readings}$$

Storing and searching that whole list at every step is not practical. So we look for a short summary of the history that keeps everything the robot needs and discards the rest (LaValle §11.2). Sections 3 and 4 give two such summaries.

3. Summary 1: the set of cells the robot could be in

> **Key point:** We can keep the set of cells that agree with every command and reading: a move turns each cell into all the cells it can reach, and a reading keeps only the cells that agree with it. With a sensor that can be wrong, the set stops shrinking.

3.1 The set, step by step

A first thing to keep in memory, shorter than the history: the cells the robot could be in. Suppose for a moment that the sensor is never wrong. The reading "door" then means "one of the door cells". So after the first reading the robot can be in:

$$\{1, 3, 7\}$$

Next, the command "move 2" moves the robot 1, 2 or 3 cells. Every cell of the set becomes all the cells it can reach, and we collect them together (the move stage of Figure 3). The symbol $\cup$ (union) means "all cells in either set":

$$\{2, 3, 4\} \cup \{4, 5, 6\} \cup \{8, 9, 0\}$$

$$= \{0, 2, 3, 4, 5, 6, 8, 9\}$$

The second reading "door" keeps only the cells of this set that are doors. The symbol $\cap$ (intersection) means "cells in both sets":

$$\{0, 2, 3, 4, 5, 6, 8, 9\} \cap \{1, 3, 7\}$$

$$= \{3\}$$

Figure 3 plays these steps, one set per row. Watch the reading cross out the wall cells, each door cell fan out to the three cells the move can reach, and the second reading cross out every cell but cell 3. Its last stages replay the second reading with a sensor that can be wrong (Section 3.2).

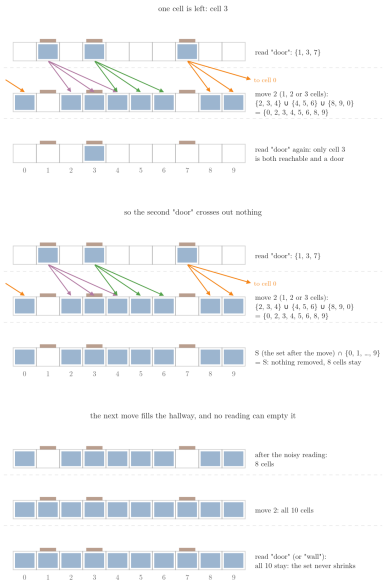


Figure 3: Animation: the hallway of 10 cells with doors (brown tabs) at cells 1, 3 and 7; blue cells are the cells the robot could be in. With a sensor that is never wrong: before any reading, all 10 cells; the reading door crosses out the wall cells and leaves {1, 3, 7}; the move of 2 cells (really 1, 2 or 3) fans cell 1 out to {2, 3, 4}, cell 3 to {4, 5, 6} and cell 7 to {8, 9, 0}, round the loop, and their union is {0, 2, 3, 4, 5, 6, 8, 9}; the second reading door crosses out every wall cell and leaves only cell 3. With the real sensor, which reads door at a wall 20 percent of the time, door can be read in all 10 cells, so the second reading removes nothing, and the next move fills all 10 cells. The last frame shows the full hallway: the set never shrinks

The perfect-sensor stages of Figure 3 end on three sets. Only cell 3 can be reached from a door by one move and is itself a door, so the robot knows exactly where it is. This set of

possible **states** (G-2314) is the **nondeterministic information state** (G-2432), also called the set-valued **information state** (G-2431) (LaValle §11.2.2). The set is far shorter than the history: at most 10 cells, however long the robot runs.

3.2 Why a set is not enough

Consider the real sensor, which reads "door" at a wall 20 percent of the time. A reading of "door" is then possible in every cell. In symbols:

- S is the set of cells the robot could be in before the reading, such as $\{0, 2, 3, 4, 5, 6, 8, 9\}$ after the move of Section 3.1;
- $\{0, 1, \ldots, 9\}$ is the set of cells where "door" can be read, which is now all 10 cells.

So the intersection $\cap$ removes nothing:

$$S \cap \{0, 1, \ldots, 9\} = S$$

The set never shrinks, however many readings arrive (the last stages of Figure 3). Crossing places out only works for a perfect sensor, for the reason shown in why noisy sensors forbid crossing places out: a wrong reading would cross out the true cell. The set also cannot say that cell 3 fits the readings much better than cell 9: both are only "possible". We need a summary that keeps a **number** for every cell, saying how strongly the readings point to it. Probabilities are that number.

4. Summary 2: a probability for every cell

> **Key point:** The **belief** (G-2433) gives every cell the probability that the robot is there, given its whole history. A reading sharpens it; a move shifts it and blurs it.

4.1 The belief, as a picture

The second thing the robot can keep in memory: instead of "possible" or "impossible", one probability for each of the 10 cells. Together they form a **probability distribution** (G-1571) (a list of every outcome with its probability, adding to 1). Figure 4 plays the running example.

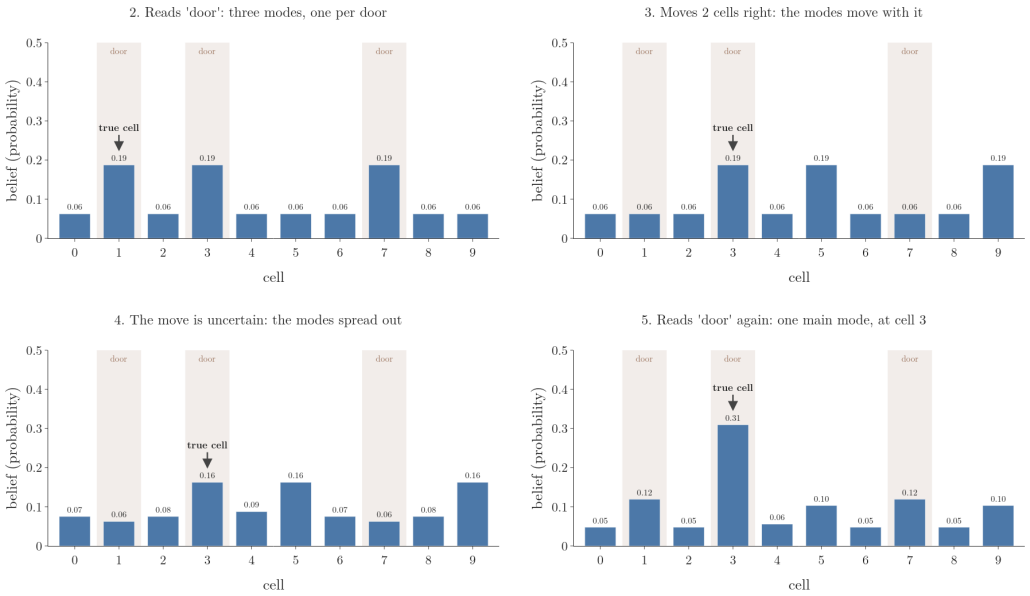


Figure 4: The belief over the 10 cells in five stages: uniform at 0.1; after reading door, three peaks (modes) of 0.19 at the doors; after moving 2 cells, the modes sit at cells 3, 5 and 9; because the move is uncertain, they spread to 0.16; after reading door again, one main mode of 0.31 at cell 3. An arrow marks the true cell

What to watch in each stage:

1. **No information yet.** The robot has no reason to prefer any cell, so each gets the same share:

$$\frac{1}{10} = 0.1$$

2. **Reads "door".** The three door cells gain probability and the seven wall cells lose it. The wall cells do not drop to zero, because the sensor reads "door" at a wall 20 percent of the time. Each door cell is now a peak of the **belief** (G-2433), a **mode** (G-1251), so the belief has three modes: it is **multimodal** (G-1275).

3. **Moves 2 cells.** Every mode moves 2 cells to the right, with the robot.

4. **The move is uncertain.** The move may have been 1 or 3 cells, so each mode spreads onto its neighbours and gets lower (0.19 becomes 0.16).

5. **Reads "door" again.** Only the mode at cell 3 sits on a door, so it rises to 0.31 while the others fall. The tallest bar is at the true cell.

Why do the door cells get 0.19 in stage 2? The sensor reads "door" 0.6 of the time at a door and 0.2 at a wall, so each door cell keeps 3 shares to a wall cell's 1, and with 3 doors and 7 walls a door cell gets 3 of the 16 shares, 0.1875, shown rounded to 0.19 (why noisy sensors forbid crossing places out). The exact rule behind this share-counting is Bayes' theorem, applied at every reading by the **correction step** (G-2448).

4.2 The definition

The robot's probability for each cell, given everything it knows, is its **belief** (G-2433). Each symbol:

- t is the time step, here 2;
- $\text{bel}(x_t)$ is the belief, the probability that the robot is in cell x_t at time t, a number from 0 to 1 (0.31 for cell 3 at $t = 2$);
- x_t is the cell at time t, any cell number from 0 to 9; the belief has one value for every possible cell;
- $p(\ldots)$ is the probability of what stands left of the bar;
- $z_{1:t}$ means all readings from time 1 to t, here $(z_1, z_2) = (\text{door}, \text{door})$ at $t = 2$;
- $u_{1:t}$ means all commands from time 1 to t, here $(\text{stay}, \text{move } 2)$;
- the bar $\mid$ is read "given", as in a **conditional probability** (G-444).

Written out:

$$\text{bel}(x_t) = p(x_t \mid z_{1:t},\ u_{1:t})$$

The right-hand side conditions on $z_{1:t}$ and $u_{1:t}$: exactly the **information state** (G-2431) of Section 2.2. So the belief is a summary of the history, made of 10 numbers however long the history grows (PR §2.3.4). At $t = 2$ in our run:

$$\text{bel}(x_2 = 3) = 0.31$$

$$\text{bel}(x_2 = 9) = 0.10$$

Unlike the set of Section 3, the belief says that cell 3 fits the readings three times better than cell 9.

4.3 A belief is not a position

The robot is always in exactly one cell. A **belief** (G-2433) of 0.19 on three cells does not mean the robot is spread over three places; it means the robot does not know which of the three it is in (Labbe ch.2). The belief describes the robot's **knowledge**, not the world. For the same reason, the arrow for the true cell and the bars in Figure 4 are drawn separately: the bars can be wrong, and the true cell does not depend on them.

4.4 Why the whole history appears in the definition

The definition conditions on every command and reading since the start, which looks like the long list we wanted to avoid. The history is there because the robot does not know its earlier cells either: if it knew its previous cell x_{t-1}, that one cell plus the latest command and reading would be enough, but it only has the history (PR §2.3.4). The derivation of the Bayes filter shows that the **belief** (G-2433) can still be computed step by step from the previous belief alone, so the robot never stores the history itself.

4.5 The predicted belief: after the move, before the reading

> **Key point:** The predicted belief is the **belief** (G-2433) after the latest command but before the latest reading. Commands lose knowledge, readings gain it, so we track the belief between the two.

The two kinds of event act in opposite directions, as moving and sensing showed. A move adds doubt, because the robot cannot be sure how far it went. A reading removes doubt, because it rules places in or out. So it is useful to look at the belief halfway through a step: after the command u_t, before the reading z_t. The belief at that halfway point is the **predicted belief** (G-2436), written with a bar. At time t:

- $\overline{\text{bel}}(x_t)$ is the predicted belief, the probability that the robot is in cell x_t after the command and before the reading, a number from 0 to 1 (0.1625 for cell 3 at $t = 2$);
- x_t is the cell at time t, any cell number from 0 to 9;
- $z_{1:t-1}$ means all readings before time t, here only $z_1 =$ door at $t = 2$;
- $u_{1:t}$ means all commands from time 1 to t, here (stay, move 2) at $t = 2$.

Written out:

$$\overline{\text{bel}}(x_t) = p(x_t \mid z_{1:t-1},\ u_{1:t})$$

The only difference from $\text{bel}(x_t)$ is that the last reading z_t is missing from the right-hand side (PR §2.3.4). Figure 5 shows both at $t = 2$.

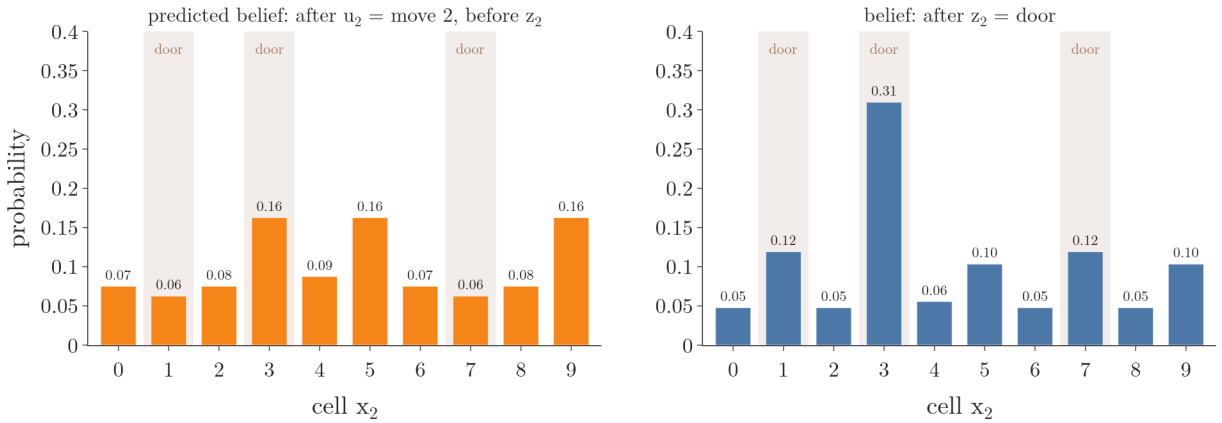


Figure 5: Left, orange: the predicted belief after move 2 and before the second reading, with 0.16 at cells 3, 5 and 9. Right, blue: the belief after the second reading door, with 0.31 at cell 3

Follow the tallest bar through time 2:

$$\text{bel}(x_1 = 1) = 0.1875$$

$$\overline{\text{bel}}(x_2 = 3) = 0.1625$$

$$\text{bel}(x_2 = 3) = 0.3095$$

The move lowered it, the reading raised it. The **Bayes filter** (G-2446) gives the two rules behind these numbers, and the **histogram filter** (G-2461) computes them cell by cell.

5. The two tables that drive the belief

> **Key point:** Two probability tables are all the robot needs: how a command moves it (the **state transition probability** (G-2434)) and what the sensor reads in each place (the measurement probability).

To update its **belief** (G-2433), its probability for every cell, the robot needs numbers for the two problems of the opening: a move of "2 cells" may really be 1 or 3, and the door sensor is sometimes wrong. Both are probability tables measured or modelled in advance.

5.1 How a command moves the robot: the state transition probability

The command "move 2" does not always move the robot 2 cells. Suppose tests on the real hallway show it moves:

Cells moved	1	2	3
Probability	0.1	0.8	0.1

Figure 6 draws the three outcomes from cell 1.

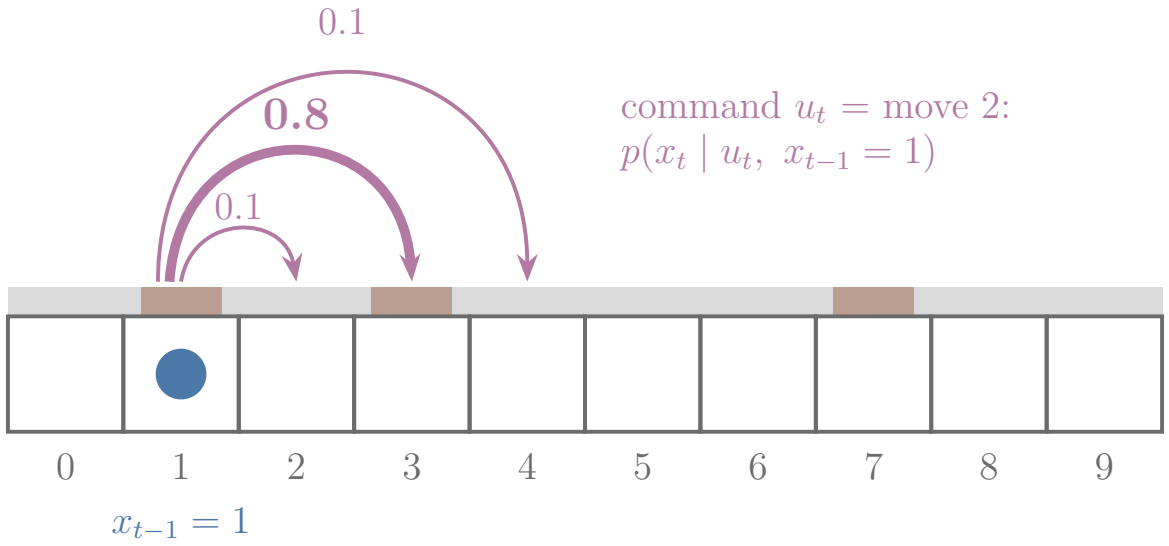


Figure 6: From cell 1, the command move 2 ends in cell 2 with probability 0.1, cell 3 with 0.8 and cell 4 with 0.1

For one move:

- x_{t-1} is the cell before the move, cell 1 in Figure 6;
- u_t is the command, "move 2";
- x_t is the cell after the move, cell 2, 3 or 4 in Figure 6;
- $p(x_t \mid u_t,\ x_{t-1})$ is the probability of landing in cell x_t, a number from 0 to 1 (0.8 for cell 3).

The probability of landing in cell x_t, given the command u_t and the previous cell x_{t-1}, is the **state transition probability** (G-2434) (PR §2.3.3):

$$p(x_t \mid u_t,\ x_{t-1})$$

From Figure 6:

$$p(x_t = 3 \mid \text{move 2},\ x_{t-1} = 1) = 0.8$$

$$p(x_t = 2 \mid \text{move 2},\ x_{t-1} = 1) = 0.1$$

The robot ends somewhere, so the three outcomes add to 1:

$$0.1 + 0.8 + 0.1 = 1$$

Why only u_t and x_{t-1}, and not the whole past? If the **state** (G-2314) is complete, the current cell and the command already contain everything that affects the next cell; how the robot got to cell 1 does not matter (the **complete state** (G-2315)). Ignoring the earlier cells is the **Markov property** (G-2316) applied to motion (PR §2.3.3).

Why is the reading z_t not in it? A reading does not move the robot. The reading helps us **guess** where the robot went, but it does not **cause** where it went. The transition probability describes the robot itself, not our guessing (PR §2.3.3).

For a real wheeled robot, x_t is a pose and the table becomes a probability density over poses: the **motion model** (G-2319). Here the hallway keeps it a small table.

A sequence of states where the next state depends only on the current one is a **Markov chain** (G-2437) (CS188 §8.1).

5.2 What the sensor reads in each place: the measurement probability

The door sensor is not perfect either. Suppose tests show:

True cell	p(reads "door")	p(reads "wall")
a door cell (1, 3, 7)	0.6	0.4
a wall cell (all others)	0.2	0.8

Figure 7 draws both columns for all 10 cells.

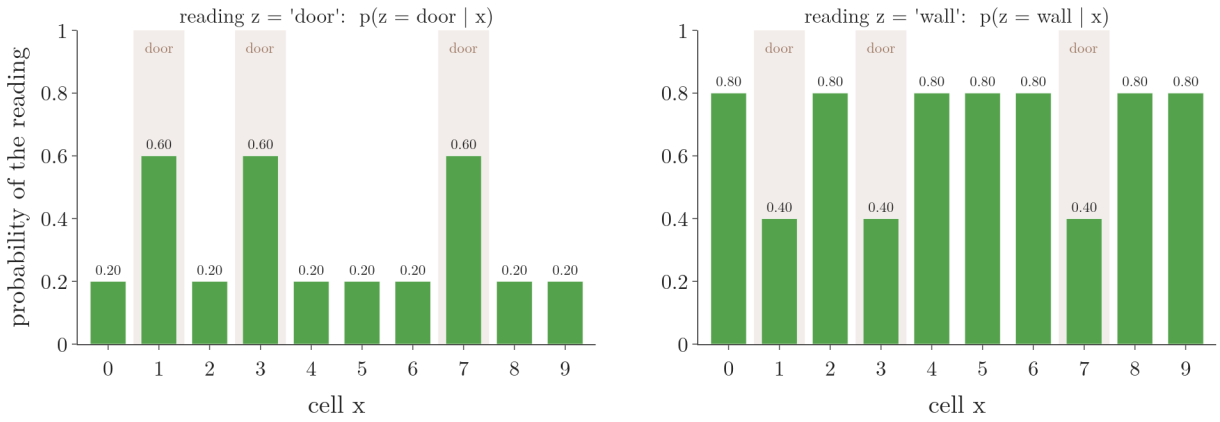


Figure 7: Left: the probability of reading door is 0.6 at cells 1, 3 and 7 and 0.2 elsewhere. Right: the probability of reading wall is 0.4 at the doors and 0.8 elsewhere

For one reading:

- x_t is the true cell at time t, such as cell 3;
- z_t is the reading at time t, "door" or "wall";

- $p(z_t \mid x_t)$ is the probability of reading z_t in cell x_t, a number from 0 to 1 (0.6 for "door" in cell 3).

The probability of a reading z_t given the true cell x_t is the **measurement probability** (G-2435) (PR §2.3.3):

$$p(z_t \mid x_t)$$

From the table:

$$p(z = \text{door} \mid x = 3) = 0.6$$

$$p(z = \text{door} \mid x = 4) = 0.2$$

In each cell, the two readings add to 1:

$$0.6 + 0.4 = 1$$

Why does it depend on x_t alone? Once the true cell is known, how the robot got there and what it read before tell us nothing more about what it will read now (PR §2.3.3). Depending on the current cell alone is the **Markov property** (G-2316) applied to readings.

Why do we write $p(z \mid x)$, reading given place, and not $p(x \mid z)$, place given reading? Because $p(z \mid x)$ is the one we can measure. We park the robot in front of a door 100 times and count how often it reads "door"; doing the same at a wall gives the other row. This direction, from cause to effect, is easy to collect; the other direction, from effect back to cause, is what Bayes' theorem computes from it (Labbe ch.2). For a real range sensor the table becomes the **beam model** (G-2377).

Together the two tables say what each cell and each reading depends on; Section 6 draws those links as one picture.

6. The whole picture: the hidden Markov model

> **Key point: States** (G-2314), the robot's cells, form a hidden **Markov chain** (G-2437), where each cell depends only on the one before and the command; each reading depends only on its own cell. Drawn as a graph, this is a dynamic Bayes network.

6.1 A small piece of maths: Bayesian networks

The two tables of Section 5 say what depends on what: the next cell on the cell and the command, the reading on the cell alone. A **Bayesian network** (G-2531) draws each variable as a circle, with arrows into it from the variables its probability depends on directly. Each circle has one table, its probability given the circles with arrows into it, and the probability of all variables together is the product of these tables (CS188 §6.3).

6.2 The robot as a dynamic Bayes network

Figure 8 draws the hallway robot this way, its hidden cells, commands and door readings, one column per time step.

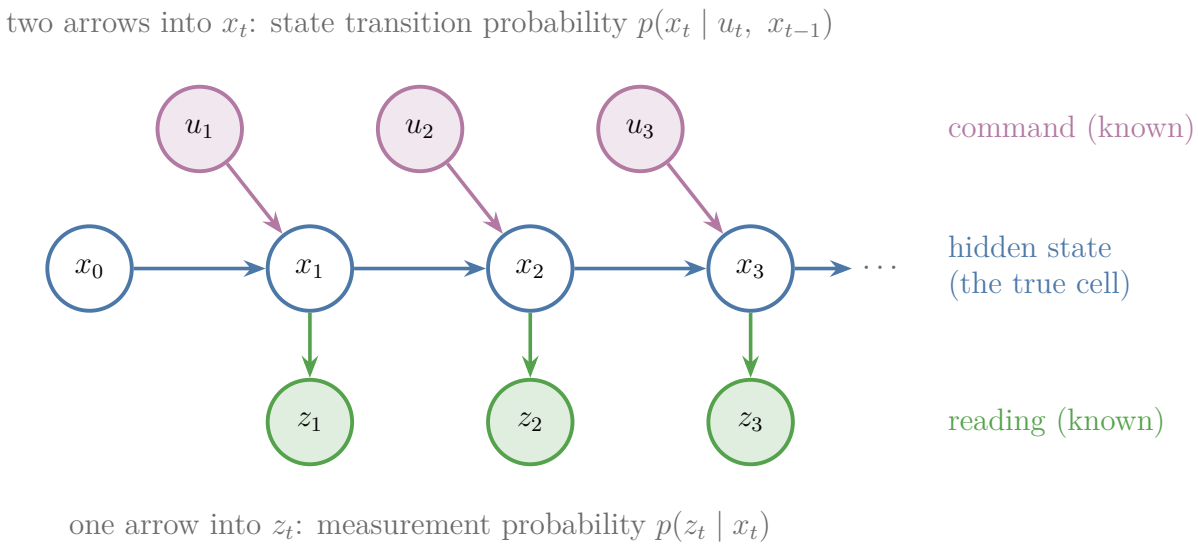


Figure 8: A chain of hidden states x0 (the cell at the start, before any command), x1, x2, x3 joined by arrows; a command u_t points into each x_t; each x_t points down to its reading z_t. Commands and readings are shaded as known, states are white as hidden

Reading the arrows:

- two arrows point into the cell x_t, from the previous cell x_{t-1} and from the command u_t: its table is the **state transition probability** (G-2434) $p(x_t \mid u_t, x_{t-1})$ of Section 5.1;
- one arrow points into the reading z_t, from the cell x_t: its table is the **measurement probability** (G-2435) $p(z_t \mid x_t)$ of Section 5.2;
- no arrow goes from the first reading z_1 to the second z_2: given the cells, one reading says nothing about another;
- the shaded circles are what the robot knows (its **information state** (G-2431)); the white ones are hidden.

A Bayesian network that repeats the same columns over time is a **dynamic Bayes network** (G-2441) (PR §2.3.3). Every robot **state** (G-2314) estimator in this chapter rests on this one drawing.

6.3 Hidden Markov model

Remove the commands from Figure 8 and a well-known model is left: a Markov chain whose **states** (G-2314) are hidden, each state producing a reading we can see. Such a chain is a **hidden Markov model** (G-2439), or HMM (CS188 §8.2). Speech recognition and weather

tracking use the same model; in robotics the hidden state is the robot's place and the reading is its sensor.

With the commands added back, the robot's model is an HMM whose transitions depend on the command; Thrun calls this the **dynamic Bayes network** (G-2441) of the robot and its environment (PR §2.3.3).

The notebook `RO-013-belief.ipynb` checks every number in this Note and lets us try other reading sequences.

7. Summary

What the robot keeps	Size	After a move	After a reading	Weakness
History (information state)	grows every step	add the command u_t	add the reading z_t	too long to keep
Set of possible cells	at most 10 cells	union of reachable cells	keep cells that fit	stops shrinking with a sensor that can be wrong; no "how likely"
Belief	10 probabilities	shift and spread	multiply up the cells that fit, then rescale	needs the two tables of Section 5

- The answer to the opening question: the robot should keep its **belief** (G-2433), 10 probabilities instead of a history of 36,000 readings after one hour, and in the running example its tallest bar after two steps, 0.31, is the true cell 3.
- The robot cannot see its true cell; it only knows its commands and readings, so every decision must be based on that history, the **information state** (G-2431).
- The history grows forever, so we keep a summary of it: the set of possible cells is short, but a sensor that can be wrong makes every cell possible, so the set stops shrinking.
- The belief keeps one probability per cell given the whole history, so it can say that cell 3 fits the readings three times better than cell 9.
- A belief describes the robot's knowledge, not its position, because the robot is always in exactly one cell.
- The **predicted belief** (G-2436) is taken after the command and before the reading, because a move adds doubt and a reading removes it.
- The **state transition probability** (G-2434) $p(x_t \mid u_t, x_{t-1})$ and the **measurement probability** (G-2435) $p(z_t \mid x_t)$ are all the robot needs, because a **complete state** (G-2315) makes the past irrelevant once the current cell is known.
- Drawn as a dynamic Bayes network (a hidden Markov model with commands), each **state** (G-2314) depends only on the previous state and its command, and each reading only on its own state, so the two tables of Section 5 are all the model needs.

The robot now knows what to keep in memory: its belief. Still missing: by definition the belief depends on the whole history, which grows forever; updating it from only the newest command and reading is the **Bayes filter** (G-2446).

8. Sources

Built from

- Udacity, Artificial Intelligence for Robotics, lesson 1 "Localization", introduction (video titled in Korean, "Total probability"; English audio), YouTube,

https://www.youtube.com/watch?v=n1EacrqyCs8

- Francisco Iacobelli, "bayesianBeliefHMMRobotics", YouTube, https://www.youtube.com/watch?v=BI
- Normalized Nerd, "Hidden Markov Model Clearly Explained! Part - 5", YouTube, https://www.youtube.com/watch?v=RWkHJnFj5rY
- NPTEL IIT Madras, "Introduction to Robotics: Introduction to Probabilistic Robotics" (#28), YouTube, https://www.youtube.com/watch?v=j7BVHy231B0
- NPTEL IIT Madras, "Introduction to Robotics: Recursive State Estimation, Bayes Filter" (#29), YouTube, https://www.youtube.com/watch?v=v0FJhNy6Fh0
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. §11.1.2 history information states, §11.1.3 plans on the information space, §11.2 derived information spaces, §11.2.2 nondeterministic information spaces. Free online: https://lavalle.pl/planning/node542.html and https://lavalle.pl/planning/node551.html (LaValle)
- DeNero, J., Klein, D. et al. *CS 188: Introduction to Artificial Intelligence*, online textbook, UC Berkeley. §6.3 Bayesian network representation, §8.1 Markov models, §8.2 hidden Markov models. https://inst.eecs.berkeley.edu/~cs188/textbook/hmms/hmm.html (CS188)
- Labbe, R. *Kalman and Bayesian Filters in Python*, ch.2 "Discrete Bayes Filter" ("Tracking a Dog" to "Noisy Sensors"). https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/02-Discrete-Bayes.ipynb (Labbe ch.2)

Other references

- Thrun, S., Burgard, W. and Fox, D. (2005). *Probabilistic Robotics*. MIT Press. §2.3.3 state transition and measurement probabilities and the dynamic Bayes network, §2.3.4 belief distributions, §2.4.2 the door example. Cited only where the free sources above say the same (PR)

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Belief (G-2433)	The robot's probability for each possible state given all its commands and readings, $\text{bel}(x_t) = p(x_t \mid z_{1:t}, u_{1:t})$; a short summary of the history that says how strongly the evidence points to each state.
Hidden Markov model (HMM) (G-2439)	A Markov chain whose states cannot be seen, where each state produces an observation that can, such as hidden robot cells producing "door" or "wall" readings; it is the model behind robot state estimation, speech recognition and many trackers.

Term	Meaning
Information state (history) (G-2431)	Everything a robot knows at time t: its starting knowledge plus every command and every reading so far, such as (stay, door, move 2, door); every decision must be based on it, because the true state is never seen.
Nondeterministic (set-valued) information state (G-2432)	The set of states that agree with every command and reading so far, such as cells {1, 3, 7} after reading "door"; a move replaces it by all reachable states, a reading keeps only the states that fit, but it says nothing about how likely each state is.
Predicted belief (G-2436)	The belief after the latest command but before the latest reading, $\overline{\text{bel}}(x_t) = p(x_t \mid z_{1:t-1}, u_{1:t})$; it separates the doubt a move adds from the knowledge a reading brings.
State transition probability (G-2434)	The probability of the next state given the command and the previous state, $p(x_t \mid u_t, x_{t-1})$, such as 0.8 for "move 2" taking the robot from cell 1 to cell 3; it tells the robot how a command changes where it is.
Markov chain (G-2437)	A sequence of random states in which the next state depends only on the current one, not on earlier ones, such as a robot repeating "move 2"; it lets a whole sequence be described by one table of transition probabilities.
Measurement probability (G-2435)	The probability of a sensor reading given the true state, $p(z_t \mid x_t)$, such as 0.6 for reading "door" in front of a door; it tells the robot how well each state explains a reading.
Bayesian network (G-2531)	A graph of random variables drawn as circles, with an arrow into each variable from the variables its probability depends on directly; each circle holds one table of its probability given those variables, and multiplying the tables gives the probability of everything together, so one huge table splits into small ones.

Term	Meaning
Dynamic Bayes network (G-2441)	A Bayesian network that repeats the same variables at every time step, such as states, commands and readings of a robot; it shows which variable depends on which over time, so a whole run's probability splits into small factors.
Motion model $p(\mathbf{x}_t \mid u_t, \mathbf{x}_{t-1})$ (G-2319)	The probability distribution of a robot's pose after a move, given its pose before and the control (command or odometry); it describes how uncertain motion is, so a filter can score possible end poses or draw them at random.
State x_t (G-2314)	The quantities a robot keeps track of at time t because they decide what happens next, such as its pose, its speeds and moving things around it; it is what localization and filtering estimate.
Probability distribution (G-1571)	A list (or curve) of every possible outcome of a random variable with its probability, adding up to 1; it says how likely each outcome is, so the chance of any event can be worked out from it.
Mode (G-1251)	The value that appears most often in the data, so it sits under the highest peak of a histogram or a distribution curve; it gives a centre for data that are not numbers, and a belief with several peaks has several modes.
Multimodal (G-1275)	Having more than one mode, that is more than one peak (two peaks: bimodal), such as a robot's belief peaking at three doors; it shows the data point to several separate values, so one mean or one best guess would hide them.
Correction step (measurement update) (G-2448)	The Bayes-filter step after a reading: multiply each state's predicted belief by the measurement probability $p(z_t \mid x_t)$ and rescale so the beliefs add to 1; it raises the states that explain the reading and sharpens the belief.
Conditional probability (G-444)	The probability of an event given that another event has happened: $P(A \mid B)$; it updates a probability with what is now known, such as the chance of reading "door" once the robot is known to be at a door.

Term	Meaning
Bayes filter (G-2446)	The algorithm that keeps a robot's belief up to date one step at a time: a prediction step after each command and a correction step after each reading; it gives the exact belief without storing the history, under the Markov assumption.
Histogram filter (discrete Bayes filter) (G-2461)	The Bayes filter run on a space cut into cells, keeping one probability per cell; sensing multiplies each cell by $p(z \mid x)$ and rescales, moving slides and blurs the list. It makes the Bayes filter computable for a continuous position.
Complete state (G-2315)	A state that, once known, makes earlier states, controls and measurements useless for predicting the future (position plus speed for a coasting robot, not position alone); it lets the robot keep only its latest estimate.
Markov property (G-2316)	The property of a sequence of states that the future depends on the past only through the present state; it lets a filter forget the history and work from the latest state alone.
Beam model (G-2377)	The measurement model of a range finder that mixes four densities for one reading (measurement noise, unexpected objects, failures, random readings) with weights that add up to 1, so no single odd reading can rule out the true pose.